Task 1 - Security Requirements Analysis and Vulnerability Identification

Course: Introduction to Secure Development Practices **Deliverable:** OWASP Top 10 summary + impact analysis for our application, and at least 5 security requirements **Application:** Pinpoint (formerly *Always Together*) - a privacy-first location-sharing app (Go + FastAPI backends, Android client, PWA web client) **Due:** 4 November 2026 - **Prepared:** 2 November 2026

---

0. What data does our application handle?

Security requirements follow from the data. Pinpoint processes:

Data class	Examples	Sensitivity
Account data	e-mail, full name, role, password **hash**	High (personal data)
Precise location	latitude/longitude, accuracy, timestamp of the user *and their friends*	**Very high** (GDPR-like personal data; reveals home, habits)
Social graph	friendships, blocks, directional location shares	High (reveals relationships)
Credentials/session material	bcrypt hashes, JWT access tokens, session tokens, CSRF tokens	Critical if leaked
Device/telemetry	Android crash reports written **locally** to `Downloads/crash_reports/`	Medium
Payment details	**none - the app handles no payment data**	-

Two conclusions drive every requirement below: (1) location is the crown-jewel data and must be protected like a credential; (2) because we do **not** handle payments, our attack surface is dominated by **account takeover** and **location disclosure**, not payment fraud.

---

1. OWASP Top 10 (2021) - summary and impact on Pinpoint

A01:2021 - Broken Access Control

Summary. Restrictions on who may access resources are not enforced correctly - IDOR, missing function-level checks, privilege escalation, tampering with JWT/cookie state.

Impact on Pinpoint. This is our highest-impact risk: every location endpoint (`POST/GET /api/v1/locations/*`, `/api/v1/locations/shares/{id}`) is a direct target. If ownership checks were client-side only, any user could read a stranger's live position - a total privacy failure of the product. The app already enforces this **server-side**: `requireAuth` middleware on every location/friendship route, friendship + share-direction checks before any recipient view, and an `requireAdmin` gate for user administration (`backend/internal/handlers/app.go`, `users.go`). The repository conformance suite and the attack-oriented API tests (`api_test.go`) assert that cross-user reads are denied. Residual risk: new endpoints must keep the "deny by default" pattern - tracked as security requirement **SR-3**.

A02:2021 - Cryptographic Failures

Summary. Sensitive data exposed through missing or misused encryption - plaintext transmission, weak algorithms, keys in code, or data unencrypted at rest.

Impact on Pinpoint. Location data in transit must never travel plaintext: production runs TLS only (HTTPS proxy in the hardened deployment config). Passwords are stored as **bcrypt** hashes with per-password salt (`internal/security/security.go`, `bcrypt.DefaultCost`), never reversible. On the Android device the session bundle is encrypted with **AES-GCM** backed by the Keystore (`SessionStore.kt`), so a lost/backup-extracted device does not yield live tokens. Crash reports contain no tokens and stay on-device unless the user shares them explicitly. Residual risks: the debug APK the team tests with is not release-signed yet, and secrets (SESSION_SECRET, DB password) must stay environment-provisioned - never in git (**SR-1**, **SR-7**).

A03:2021 - Injection

Summary. Untrusted data interpreted as code/command/SQL - SQL/NoSQL injection, XSS, command injection, LDAP/XPath injection.

Impact on Pinpoint. All SQL goes through parameterised queries via `pgx` / `sqlc`-style statement builders - no string-concatenated SQL exists in the Go store or the SQLAlchemy ORM layer, which removes classic SQL injection. On the web client the main XSS surface is `innerHTML`; the client escapes every interpolated value through an `esc()` helper before assignment and prefers `textContent` for message rendering (`web/public/js/main.js`), which keeps reflected/stored XSS out of the friend and share lists. Backend validation of e-mail/password fields (`validateLogin`, `validateRegister`) rejects malformed input before it reaches the store. Residual risk: future UI code must keep using `esc()` / `textContent` - a lint/review rule, tracked under **SR-8/SR-10**.

A04:2021 - Insecure Design

Summary. Missing or misused security controls inherent to the design - no threat model, absent rate-limit design, trust of client-side logic.

Impact on Pinpoint. The threat model shapes whole features: profiles are **private by default**, location flows are **directional** (A shares with B does not imply B shares with A), sharing is opt-in per friendship, and precision can be downgraded to "approximate". Authentication is designed to avoid user enumeration: login failures return a **generic** `AUTHENTICATION_FAILED` (401) whether the account is unknown or the password wrong. The combined sign-in/sign-up flow added in November was designed so account creation still goes through the *same* server-side validation and rate limits as the admin provisioning path - the design never trusts the client to enforce anything (**SR-3**, **SR-6**).

A05:2021 - Security Misconfiguration

Summary. Default accounts, open cloud storage, verbose errors, permissive CORS/CSP, missing hardening headers.

Impact on Pinpoint. Demo seeds (`alice/bob/admin`, tester `user@user.user`) exist only behind `SEED_DEMO_DATA=true` and the seeded administrator ships with `MustChangePassword=true`; production must run with the flag off or rotated passwords (**SR-7**). The web client ships a restrictive **Content-Security-Policy** that allow-lists exactly one tile origin (`maps.wikimedia.org`) in `img-src`; CORS is an explicit origin allow-list with credentials allowed only for configured origins (`corsMiddleware`), never `*`. Debug/verbose mode is off by default. Open findings: Dependabot currently reports vulnerabilities on the default branch - an active misconfiguration-management backlog item (**A06/SR-8**).

A06:2021 - Vulnerable and Outdated Components

Summary. Using components with known vulnerabilities - outdated libraries, unsupported frameworks, untrusted downloads.

Impact on Pinpoint. Concrete and current: GitHub Dependabot reports **20 open vulnerabilities (8 critical, 4 high, 8 moderate)** on the default branch (notified on our 2 November push). Backend mitigations already in CI: `pip-audit` on Python dependencies, `bandit` static analysis, `go vet` / `go build` gates. Mobile: MapLibre Native (`org.maplibre.gl:android-sdk:11.5.1`, Maven Central, BSD-2) is pinned in the version catalogue; we deliberately replaced the unmaintained `retrofit2-kotlinx-serialization-converter` with a small **hand-written** `JsonConverterFactory` instead of shipping a dead dependency; the Gradle wrapper is committed and pinned (8.14). Remediation of the Dependabot queue is tracked as **SR-8** with a per-PR update rule.

A07:2021 - Identification and Authentication Failures

Summary. Weak authentication: credential stuffing, brute force, weak passwords, poor session handling, session fixation.

Impact on Pinpoint. Account takeover would expose the attacker to live friends' locations, so this class is critical for us. Defences implemented: minimum 8-character password policy enforced **server-side** (both `/auth/login` and the new `/auth/register`), bcrypt verification, **per-IP rate limiting** on login/register, **account lockout** after repeated failures with timed unlock, generic failure responses (no enumeration), session expiry plus idle timeout, **double-submit CSRF** tokens on cookie-authenticated writes, and `Secure` / `HttpOnly` / `SameSite` cookie attributes. The Android client never persists plaintext passwords (state is cleared after login) and stores tokens only in AES-GCM-encrypted storage. Residual risk: no MFA yet - noted in **SR-4** as a future requirement.

A08:2021 - Software and Data Integrity Failures

Summary. Trusting updates, CI/CD pipelines, plugins, and serialized data without integrity verification; insecure deserialization.

Impact on Pinpoint. Integrity risks concentrate in the pipeline: all merges go through GitHub Actions CI (Go tests incl. Postgres conformance, Python lint/ `bandit` / `pip-audit` /migrations/tests, Android compile + JVM tests, web ES-module syntax checks) before landing on `main`. Dependencies are resolved from official registries only (Maven Central, PyPI, npm via lockfiles/module sums). On-device, the API responses are deserialized with `kotlinx.serialization` (typed DTOs, `ignoreUnknownKeys`) rather than reflective parsers, which narrows deserialization gadget risk. Honest gaps: Actions are pinned by major tag (`@v4`) rather than commit SHA, and release APKs are not yet signed with a release key - both tracked in **SR-8/SR-10**.

A09:2021 - Security Logging and Monitoring Failures

Summary. Breaches go undetected because events are not logged, monitored, or alerted on.

Impact on Pinpoint. Without logs we could not detect location scraping or credential-stuffing runs. Implemented: login attempts (success/failure counts) and account-lock events recorded server-side (`RecordLoginAttempt`, lockout bookkeeping), an `audit` trail for auth/registration/rate-limit events in the FastAPI stack, and structured server logs. The Android `CrashLogger` writes tagged `Pinpoint` log lines and local crash files - deliberately **no third-party crash/analytics SDK** (0-trackers baseline), so crash data cannot leak to a vendor. Gaps acknowledged: no centralized alerting yet (log shipping depends on the Render host) and log retention is unbounded - future work under **SR-9**.

A10:2021 - Server-Side Request Forgery (SSRF)

Summary. The server is tricked into making requests to internal/unexpected destinations (cloud metadata endpoints, internal services).

Impact on Pinpoint. Today the backends make **no outbound requests to user-supplied URLs** - the only outbound traffic is the mobile/web clients fetching map tiles from a fixed, code-configured origin. That makes current SSRF exposure low, *but* the risk appears the moment we add features like webhooks, avatar-from-URL import, or import of external safety feeds. The standing rule (requirement **SR-3** extension): any future server-side fetch must go through an explicit destination allow-list with no redirects to private ranges.

---

2. Security requirements (11 defined, at least 5 required)

Each requirement states the rule, why our data demands it, and how completion is verified - verification is deliberately *testable* so it can be checked in CI.

#	Requirement	Rationale (our data)	Verification
SR-1	**Encrypt data in transit.** All client<->API and API<->DB traffic runs over TLS in production; no plaintext HTTP endpoints beyond local development.	Precise location and credentials must not be observable on the network.	HTTPS-only deployment config (hardened proxy); dev-only HTTP documented in `docs/DEPLOYMENT.md`
SR-2	**Protect credentials at rest.** Passwords stored only as salted bcrypt hashes; minimum 8 characters enforced server-side; plaintext passwords never persisted, logged, or returned.	Account takeover exposes live location of the user *and* their friends.	bcrypt in `internal/security`; API tests assert 422 on weak passwords and generic 401 on failure; no secrets in repo (CI + review)
SR-3	**Server-side, deny-by-default access control.** Every read/write of location, friendship and admin data is authorised on the server (session + ownership + friendship direction + role); client checks are UX only.	Location disclosure to an unauthorised peer is the worst possible product failure.	`requireAuth` / `requireAdmin` middleware; attack-oriented API tests (cross-user access denied); conformance suite on both DB drivers
SR-4	**Session security.** Expiring sessions with idle timeout, CSRF double-submit protection on state-changing cookie requests, `Secure` / `HttpOnly` / `SameSite` cookies, AES-GCM + Keystore encryption of stored session material on Android, secure token clearing on logout.	Stolen session ~ full account takeover without needing the password.	Session/CSRF tests in `api_test.go`; `SessionStore.kt` (AES-GCM); (future: MFA)
SR-5	**Location privacy by design.** Private-by-default profiles, explicit opt-in directional sharing, selectable precision (exact/approximate), latest-position-only retention, no location in logs, no trackers/telemetry ever, visible OSM/ODbL attribution.	Location is the most sensitive datum we hold and is directly GDPR-class personal data.	ADR *store-latest-location-only*; `mobile/THIRD_PARTY_NOTICES.md` (0 trackers); attribution visible on map screens; CSP tile allow-list

#	Requirement	Rationale (our data)	Verification
SR-6	**Rate limiting and anti-abuse.** Per-IP limits on login/register, account lockout with timed unlock, generic auth errors (no user enumeration), admin-only provisioning for privileged roles.	Credential stuffing directly targets location data; bots could enumerate members.	`RateLimiter` + lockout in Go API; `ratelimit` dependency in FastAPI; tests for 429 and lockout paths
SR-7	**Secrets and configuration hygiene.** No secrets or real user data in the repository (including tests); all credentials via environment/dashboards; demo/seed accounts only behind explicit flags and flagged for password change; shared credentials rotated after use.	Leaked DB/session secrets collapse every other control.	`.gitignore` for `.env*`; `SESSION_SECRET` generated per deployment; seed flag `SEED_DEMO_DATA`; Dependabot secret scanning
SR-8	**Dependency and component security.** CI runs `pip-audit`, `bandit`, `go vet`; Dependabot alerts triaged within one PR cycle; third-party libraries pinned via catalogues/wrappers; new mobile dependencies added to `THIRD_PARTY_NOTICES.md` in the same PR.	Known-vulnerable components are the currently *live* finding (20 Dependabot alerts).	CI jobs (backend, mobile, web); `libs.versions.toml`; `THIRD_PARTY_NOTICES.md`
SR-9	**Auditability and incident response.** Security-relevant events (login success/failure, lockout, registration, rate-limit denials, admin actions) are recorded with actor/time/outcome; crash reports stay local and user-shareable; log retention and alerting defined before production.	We must be able to detect and investigate location scraping or account-takeover attempts.	`audit` trail (FastAPI), login-attempt records (Go); local `Downloads/crash_reports/` files; (future: log shipping + alert thresholds)
SR-10	**Security changes ship with fail-closed tests.** Any change to auth, authorisation or encryption requires a regression test *and* a test proving the failure mode closes (e.g., rejected registration must create no account; duplicate registration must not take over an account).	Prevents "fixes" that silently reopen the hole they patched.	Contribution rule in `CONTRIBUTING.md`; implemented for `/auth/register` (`TestRegister*FailClosed`)
SR-11	**Minimise data.** Store only the latest location per share, never collect what a feature does not need, no analytics/ads SDKs, no third-party crash reporters.	Reduces both breach impact and regulatory scope.	ADR *store-latest-location-only*; privacy baseline in `docs/PRIVACY.md`; 0-tracker mobile baseline

> The five mandatory requirements are covered by **SR-1 (encryption in transit), SR-2 (credential protection), SR-3 (access control), SR-4 (session security), SR-5 (location privacy)**; SR-6...SR-11 extend them with operational controls.

---

3. References

1. OWASP Top 10 2021 - https://owasp.org/Top10/
2. OWASP Cheat Sheet Series (Authentication, Access Control, Logging) - https://cheatsheetseries.owasp.org/
3. Repository evidence: `backend/internal/handlers/*` (middleware, routes, tests), `backend/internal/security/security.go`, `docs/SECURITY.md`, `docs/adr/*`,

`mobile/THIRD_PARTY_NOTICES.md`

4. CI pipeline: `.github/workflows/ci.yml` (Go, Python, Android, web jobs)